4. How Machines Learn, Part 2

Gradient descent and generalization

Week 4 · CE/ARCE 2XX AI Literacy for Civil and Architectural Engineering



COLLEGE OF ENGINEERING
Civil & Architectural Engineering & Mechanics

Agenda

Today's plan

1. Quiz on last week (5–8 min)
2. Gradient descent: walking downhill, and the learning rate
3. More features, and why we scale them
4. Will it work on new data? Train/test, under- and overfitting, metrics
5. From scratch to scikit-learn — and the recipe behind all modern AI
6. This week's lab day, and Homework 2

Recap

Last week in 60 seconds

1 A model

A line $\hat{y} = w \cdot x + b$, with parameters w and b.

2 A cost

J(w, b) = average squared error ÷ 2. Lower is better.

3 A bowl

J over all (w, b) is a bowl; the best line is at the **bottom**.

4 A verdict

Best w/c line (all data): MAE ≈ 12 MPa, $R^2 \approx 0.25$ — right trend, big errors.

Learning objectives

By the end of today, you can…

- ✓ Explain gradient descent in words and apply the update rule $w \leftarrow w - \alpha \cdot \partial J/\partial w$
- ✓ Choose a learning rate α by reading a learning curve
- ✓ Use several features at once and explain why we scale them
- ✓ Split data into training and test sets, and spot underfitting and overfitting
- ✓ Report MAE, RMSE, and R^2, and fit a model in a few lines of scikit-learn

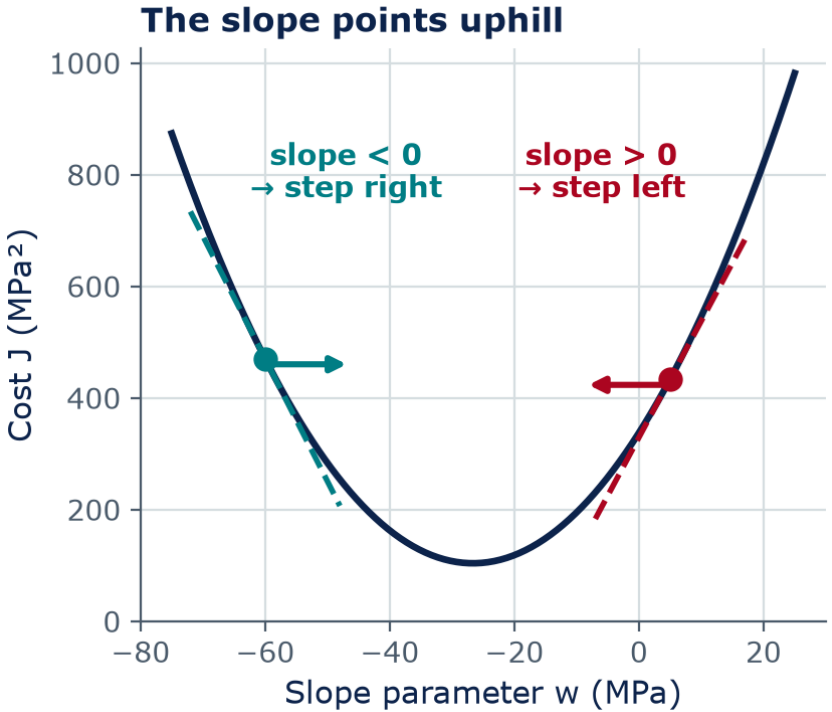
01

Gradient descent

Walking downhill in the fog

Walking downhill in the fog: the slope tells you which way to go

- You stand on a foggy hillside and want the valley floor. You can only feel the ground under your feet.
- Feel the **slope**, take a step **downhill**, repeat
- The slope of the cost curve is its **derivative** ∂J/∂w
- Slope > 0 → decrease w. Slope < 0 → increase w. Slope = 0 → you are at the bottom.



The update rule: step against the slope, scaled by α

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w} \qquad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

←	"becomes": replace the old value with the new one
α	learning rate (alpha): how big each step is
∂J/∂w	slope of the cost in the w direction
∂J/∂b	slope of the cost in the b direction
both	compute both slopes first, then update w and b together

For a line, the slopes are averages of the errors

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)} \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)$$

- **∂J/∂b** = the average error. If the line predicts too high on average, lower b.
- **∂J/∂w** = the average error × x. Errors at large x push w the most.
- You do not need to derive these — the ½ in J makes them simpler. You will code them in Lab 04.

Gradient descent from scratch: ten lines of NumPy

```
# x: w/c ratios (m values), y: strengths in MPa
w, b = 0.0, 0.0                  # start anywhere
alpha = 0.5                      # learning rate
for step in range(400):
    y_hat = w * x + b            # predictions (MPa)
    err = y_hat - y              # errors (MPa)
    dj_dw = np.mean(err * x)     # slope in w
    dj_db = np.mean(err)         # slope in b
    w = w - alpha * dj_dw        # update both,
    b = b - alpha * dj_db        # using old slopes
print(w, b)
```

- Each loop = one **iteration** (one step downhill)
- `np.mean` does the **(1/m) Σ** part
- Both slopes are computed **before** w and b change
- After 400 steps: w ≈ −26.6, b ≈ 55.8 — the best line from last week

One step of gradient descent, by hand

4 min

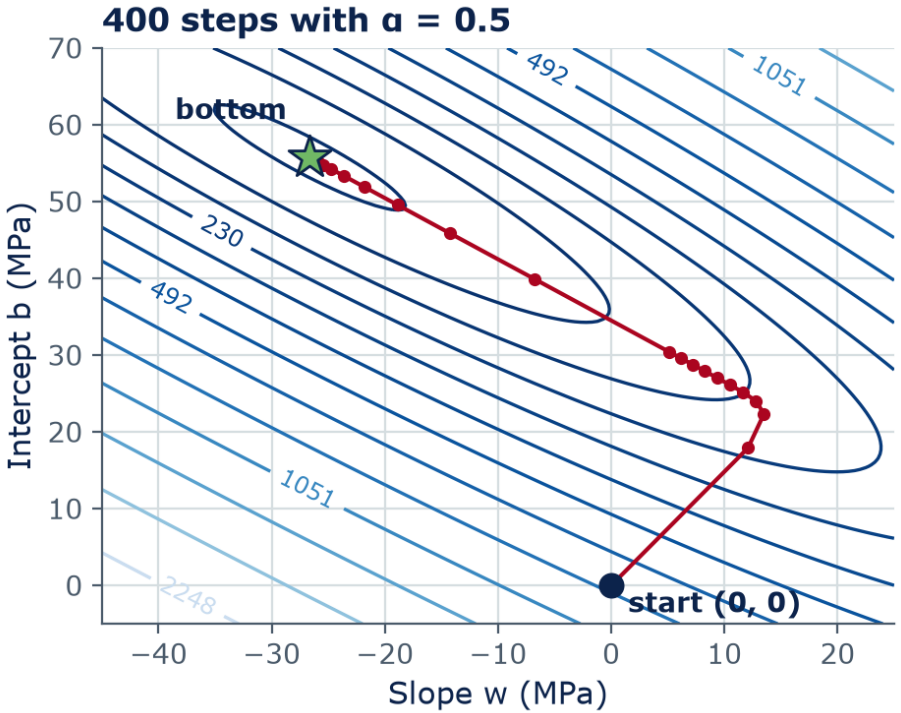
Pairs

- Two mixes: **A** x = 0.5, y = 44 MPa · **B** x = 1.0, y = 32 MPa
- Start at **w = 0, b = 40**, learning rate **α = 0.5**
- Compute both errors $\hat{y} - y$, then $\partial J/\partial w$ and $\partial J/\partial b$ (m = 2)
- Update w and b. Did the cost J go down?

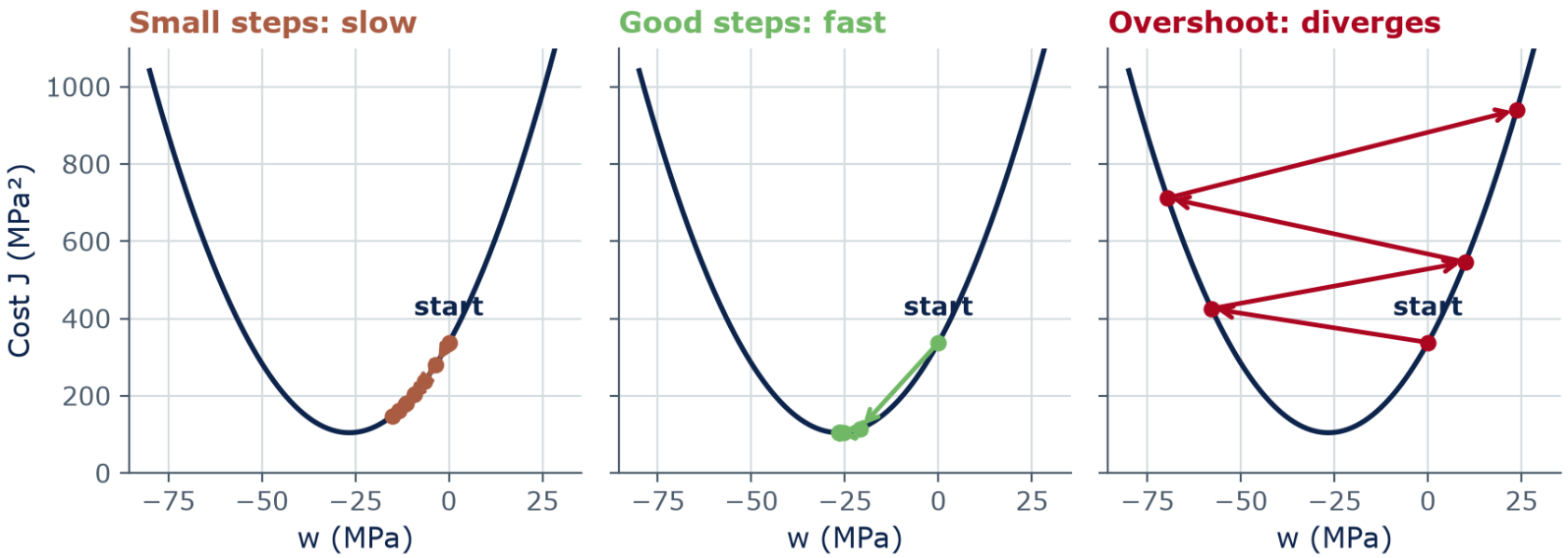
Check: errors −4 and +8 → $\partial J/\partial w = (-4 \cdot 0.5 + 8 \cdot 1.0)/2 = 3$, $\partial J/\partial b = (-4 + 8)/2 = 2$ → w = −1.5, b = 39 → J falls from 20 to about 15.8 MPa²

On the contour map, gradient descent walks to the bottom

- Each dot = (w, b) after one step; the path starts at (0, 0)
- Steps are big where the bowl is steep, small near the bottom
- The path bends: it first drops into the long valley, then crawls along it
- Lab 04: you will draw this path for your own learning rates



The learning rate α sets the step size: too small, about right, too large

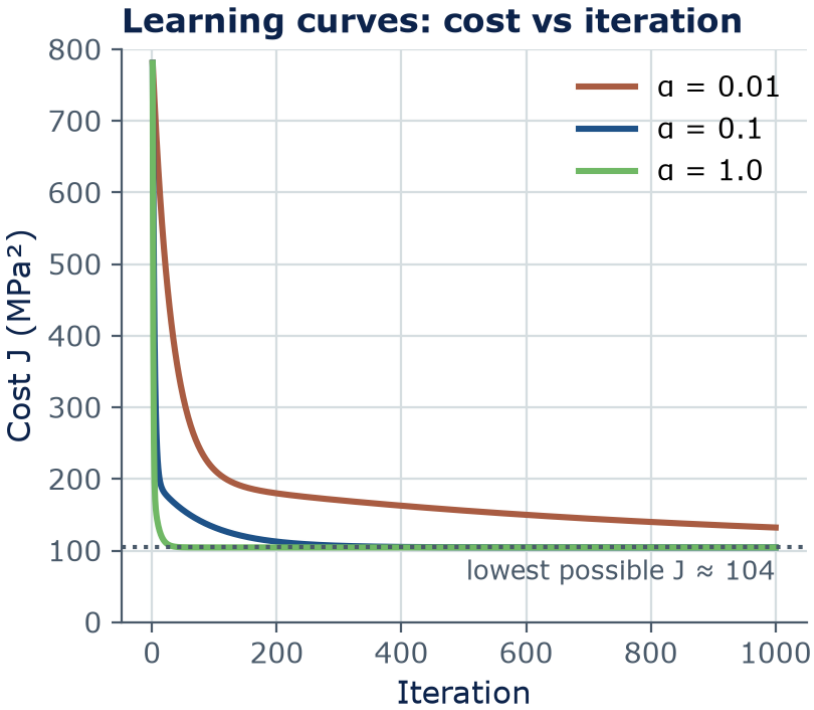


Cost J vs w with **b held fixed** (6 steps from w = 0). When w and b both move (Lab 04), the limit on this data is **≈1.25**.

Learning rate

A learning curve shows whether training is working

- Plot cost J after each iteration — it should **go down and flatten**
- α = 0.01: still falling slowly after 1,000 steps
- α = 1.0: flat within about 100 steps → converged
- α = 1.3 (w and b both moving): J explodes — over a billion MPa² after 100 steps
- Rule of thumb: try α = 0.001, 0.01, 0.1, 1 and keep the fastest one that still goes down



Concept check

Read the learning curve

You run gradient descent and the cost J gets LARGER every iteration. What should you do first?

A Run more iterations

B Make the learning rate α smaller

C Make the learning rate α larger

D Add more features

Concept check

Read the learning curve — answer

You run gradient descent and the cost J gets LARGER every iteration. What should you do first?

A. Run more iterations

B. Make the learning rate α smaller

C. Make the learning rate α larger

D. Add more features

A rising cost means each step overshoots the bottom: reduce α (e.g., by a factor of 10).

02

More features

Using all eight inputs — and scaling them

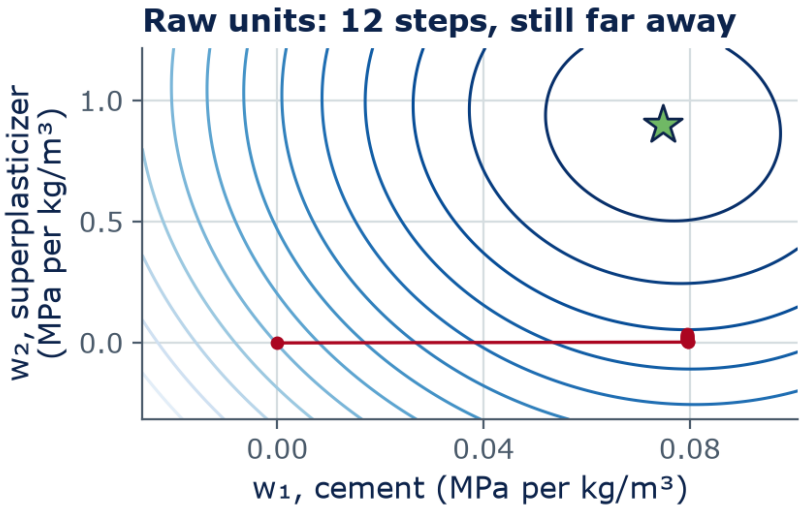
More features

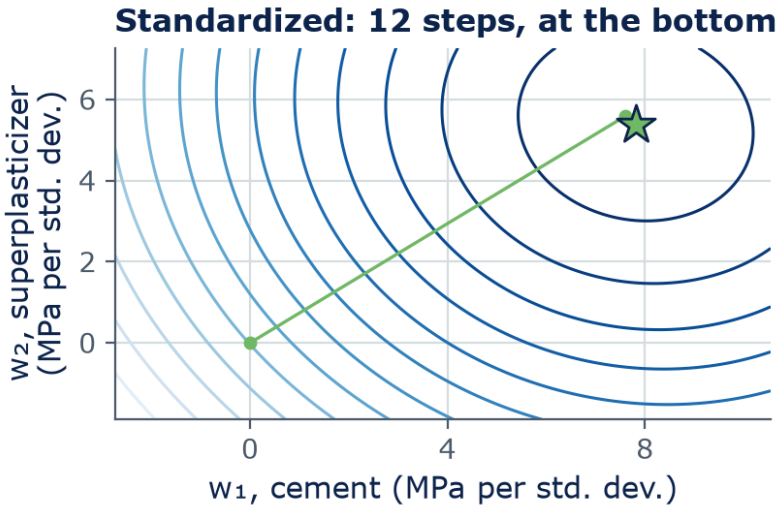
With many features, each one gets its own weight

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_8 x_8 + b = \mathbf{w} \cdot \mathbf{x} + b$$

$x_1 \ldots x_8$	the 8 inputs: 7 ingredients (kg/m³) and age (days)
$w_1 \ldots w_8$	one weight per feature (MPa per unit of that feature)
$\mathbf{w} \cdot \mathbf{x}$	dot product: multiply matching pairs and add them up
b	bias (intercept), MPa
same recipe	same cost J, same gradient descent — one slope per weight

Features on very different scales slow gradient descent down





Standardize each feature:
z = (x − mean) ÷ standard deviation

scikit-learn: `StandardScaler`
fit it on the **training** data only

03

Will it work on new data?

Train/test splits, underfitting, overfitting, metrics

Generalization

Hold back data the model never sees: the test set

All 1,030 concrete mixes, shuffled at random

Training set · 80% · 824 mixes
the model learns w and b from these

Test · 20%
206 mixes

1 · Fit

Learn the parameters on the training set only

2 · Tune

Pick settings (α, degree) with a validation set carved from training

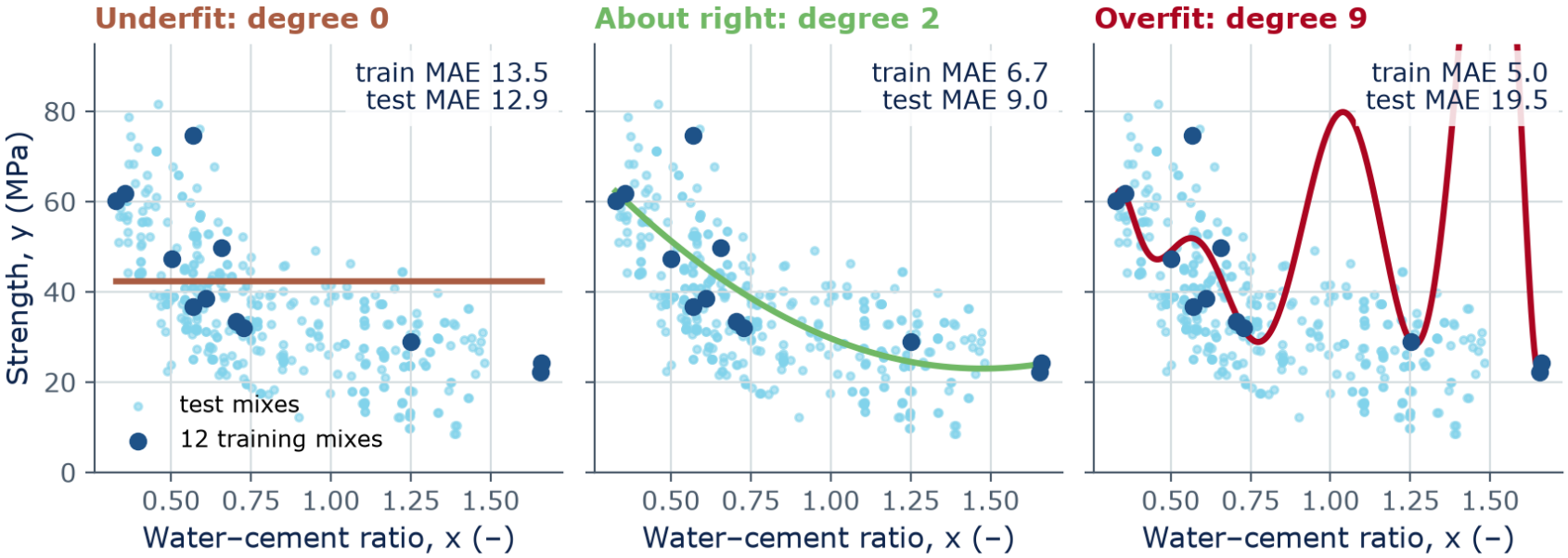
3 · Test once

Report errors on the test set — the honest estimate

Never train on the test set: grading a model on its own homework flatters it

Generalization

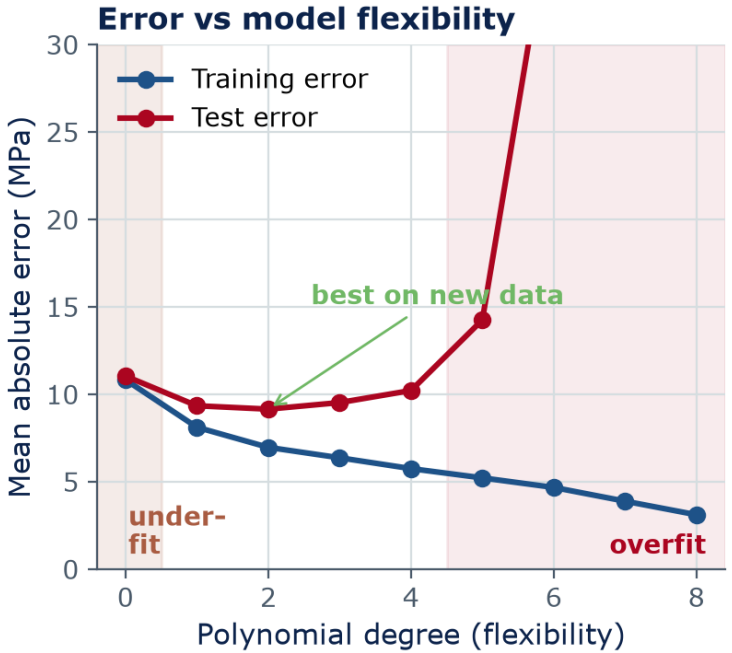
Too simple underfits; too flexible overfits



Polynomials in w/c fitted to 12 random 28-day mixes (dark dots); light dots are other 28-day mixes used as a test set. Errors in MPa.

Training error always falls with flexibility; test error does not

- **Underfitting:** high error on training **and** test — the model is too simple
- **Overfitting:** low training error, **high test error** — the model memorized noise
- The sweet spot is where test error is lowest
- Fixes for overfitting: more data, a simpler model, regularization, early stopping (Week 5)



Average of 100 random draws of 12 training mixes

Metrics

Three numbers to report: MAE, RMSE, and R²

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}\left|\hat{y}^{(i)} - y^{(i)}\right|$$

MAE (MPa)

Typical size of a miss

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2}$$

RMSE (MPa)

Like MAE, but big misses count more

$$R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2}$$

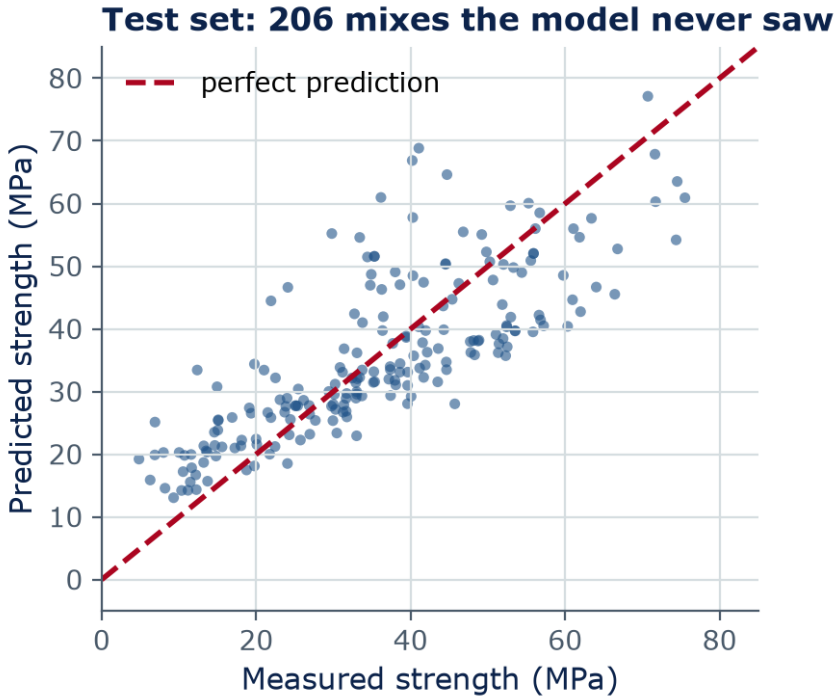
R² (no units)

1 = perfect; 0 = baseline (always predict the average)

Metrics

All eight features: test error drops by a third

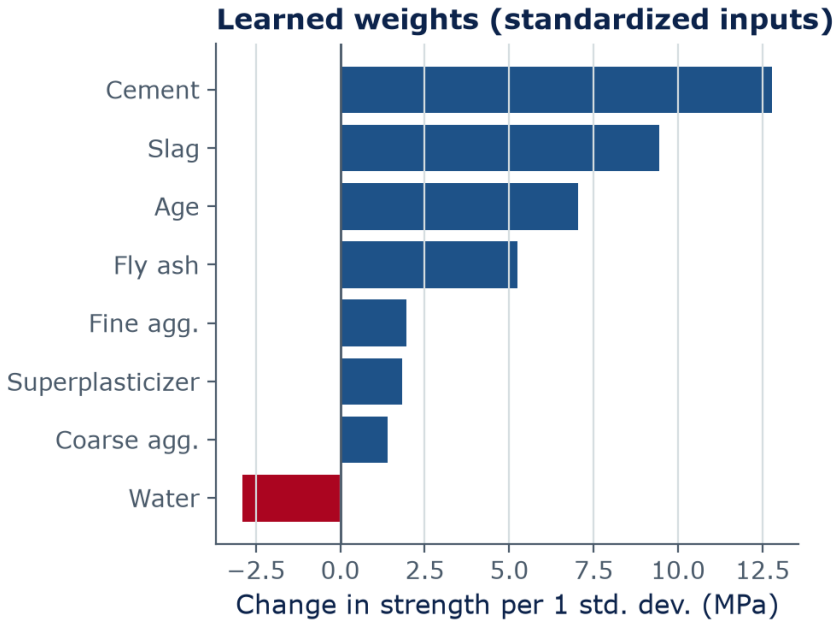
- Test set (206 unseen mixes): **MAE 7.7 MPa**, **RMSE 9.8 MPa**, **R² 0.63**
- w/c ratio alone (same test set): MAE 11.3 MPa, RMSE 13.7 MPa, R² 0.27
- Training R² 0.61 ≈ test R² 0.63 → no sign of overfitting
- Points spread widely around the diagonal: still **not good enough** to replace cylinder tests



Metrics

Read the learned weights — carefully

- Each bar = change in strength (MPa) when that input rises by **one standard deviation**, others fixed
- Cement, slag, and age push strength up the most — as engineers expect
- Water's bar is short (-2.9 MPa per SD) because water varies little here (SD ≈ 21 kg/m³ vs 107 for cement); per kg, its effect matches Abrams' law
- These are **associations in this dataset**, not proven causes



Would you sign off on this model?

4 min

Think · Pair · Share

- A supplier proposes: use the 8-feature model (test MAE ≈ 8 MPa) instead of 28-day cylinder tests on a parking garage
- With a partner: list **two reasons to say no** and **one good use** of the model
- Be ready to share

Think about: the size of the error vs the specified strength · mixes outside the data range · who is responsible if the slab is weak

04

From scratch to scikit-learn

…and the recipe behind neural networks and LLMs

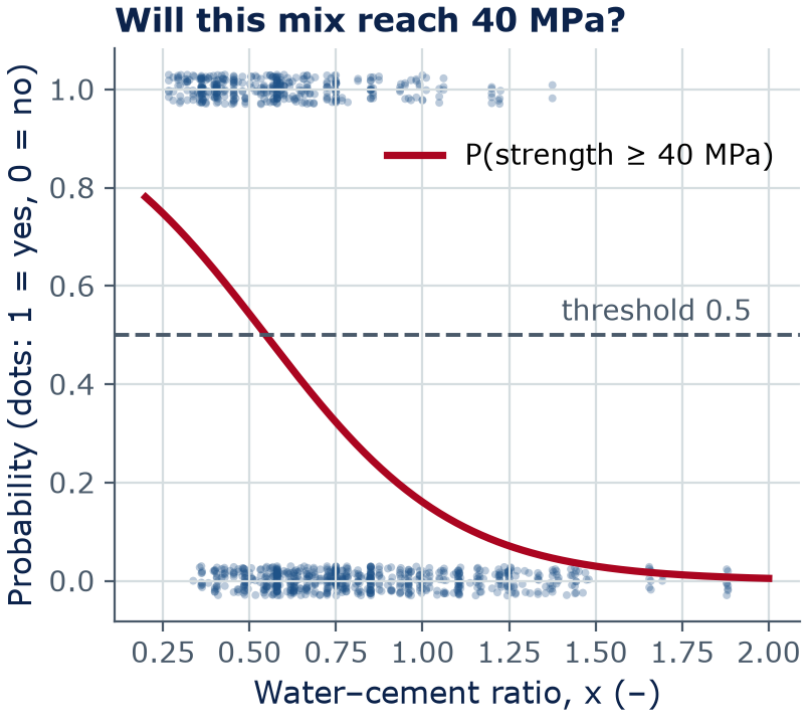
The whole workflow in scikit-learn: split, scale, fit, score

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
feature_cols = ["cement_kg_m3", "slag_kg_m3", "fly_ash_kg_m3", "water_kg_m3",
                "superplasticizer_kg_m3", "coarse_agg_kg_m3", "fine_agg_kg_m3", "age_days"]
X, y = df[feature_cols], df["strength_mpa"]       # 8 features; target (MPa)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler().fit(X_train)            # learn mean and std (training only)
model = LinearRegression().fit(scaler.transform(X_train), y_train)
y_pred = model.predict(scaler.transform(X_test))
print("MAE:", mean_absolute_error(y_test, y_pred), "MPa")   # about 7.7 MPa
print("R2: ", r2_score(y_test, y_pred))                     # about 0.63
```

Classification (if time)

From numbers to yes/no: logistic regression in one slide

- Same weighted sum $z = w \cdot x + b$, squeezed by the **sigmoid** $\sigma(z) = 1 / (1 + e^{-z})$ into 0 to 1
- Output = a **probability**; a **threshold** (often 0.5) turns it into yes/no
- Here: P(mix reaches 40 MPa) falls as w/c rises; it crosses 0.5 at w/c ≈ 0.55
- Trained the same way: a cost (log loss) + gradient descent
- Uses: crack / no crack, poor / fair deck rating (Lab 11)



The big picture

The same recipe trains neural networks and large language models

	Linear regression (Weeks 3–4)	Neural network (Weeks 5–6)	Large language model (Weeks 7–8)
Model	$\hat{y} = w \cdot x + b$	Layers of weighted sums + nonlinear functions	A transformer that predicts the next word piece
Parameters	2 to 9	Hundreds to millions	Billions (Qwen2.5-0.5B: about 0.5 billion)
Cost	Squared error	Squared error or log loss	Log loss on the next word piece
How it learns	Gradient descent	Gradient descent + backpropagation	Gradient descent + backpropagation, on many GPUs

Lab day: gradient descent, from scratch and with scikit-learn

LAB

04

Bring a charged laptop

Open the notebook from the course GitHub page

- Code the gradients and the descent loop for one feature (w/c ratio)
- Plot the path and learning curves; try α = 0.001, 0.01, 0.1, 1.0, and 1.3; scale features
- All 8 concrete inputs with scikit-learn and a train/test split
- MAE, RMSE, R^2 against a **baseline** (always predict the average); predicted vs actual; weights
- **Exit ticket:** 3 short questions you answer alone, without AI, in the last 10 minutes of lab

Homework 2

Homework 2 is posted: Fit, Measure, Descend

Due **end of Week 5** — Sunday 11:59 pm on Brightspace. Individual.

AI-FREE

Part A — By hand

Cost J and one gradient-descent step for three concrete mixes

AI-ASSISTED

Part B — From scratch

Heating load from relative compactness; a learning-rate experiment

AI-ASSISTED

Part C — scikit-learn

8 design features, train/test split; MAE, RMSE, R^2

AI-ASSISTED

Part D — Memo

Can a quick model replace a full energy simulation at the concept stage?

Wrap-up

Key takeaways

1. Gradient descent: step downhill with $\mathbf{w \leftarrow w - \alpha \cdot \partial J / \partial w}$ (same for b), and repeat
2. The learning rate α must be tuned: too small is slow, too large diverges — check the learning curve
3. Several features: $\hat{y} = w \cdot x + b$ with one weight each; standardize features so descent is fast
4. Judge a model on a held-out test set: watch for underfitting and overfitting; report MAE, RMSE, R^2
5. Model + cost + gradient descent is the recipe behind neural networks and LLMs

Learn more

Readings and resources

- **Google ML Crash Course: Gradient descent (and the Hyperparameters page after it)** — interactive, about 30 min
- **Google ML Crash Course: Datasets, generalization, and overfitting** — read “Dividing the original dataset” and “Overfitting”
- **3Blue1Brown: Gradient descent, how neural networks learn** — video, 21 min — a bridge to Week 5
- **scikit-learn user guide: Linear models** — reference for the lab